

Prompt:

8/17/2026, 9:47:24 PM

lanjut Tahap F

Response:

8/17/2026, 9:47:25 PM

Tentu. **Tahap F adalah fondasi eksperimental seluruh sistem kita.** Kalau Tahap A-E adalah membangun "otak" strategi, maka Tahap F adalah membangun **dunia virtual yang cukup realistis untuk menguji otak tersebut.**

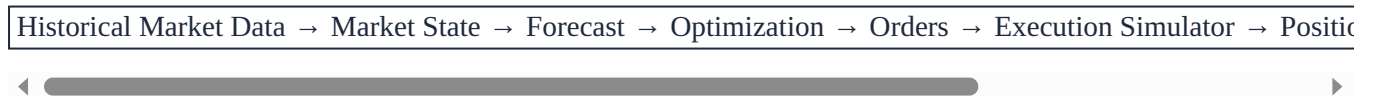
Dan saya ingin menekankan satu perubahan paradigma:

Kita tidak lagi membuat backtest sebagai fungsi sederhana `signal → return`. Kita membuat **event-driven market simulator** yang mereplikasi urutan kejadian: market data → signal → order → fill → position → risk management → cancellation/modification → exit → PnL.

Dengan begitu, nanti kita benar-benar bisa menguji apakah QAOA/quantum optimizer memberikan keuntungan dibandingkan metode klasik.

Tahap F — Event-Driven Backtesting & Simulation Engine

Arsitektur besar kita sekarang:



dan semuanya berjalan secara temporal:

$$t_0 < t_1 < t_2 < \dots < t_T.$$

F.1. Mengapa backtest biasa tidak cukup?

Bayangkan kita mempunyai signal:

LONG BTC

pada:

$$t = 10 : 00.$$

Harga:

$$P_{10:00} = 60,000.$$

Strategi mengatakan:

$$Entry = 59,950.$$

Kemudian harga bergerak:

$$59,980 \rightarrow 59,960 \rightarrow 59,950 \rightarrow 59,900 \rightarrow 60,100.$$

Backtest sederhana mungkin melihat candle:

$$Low = 59,900,$$

lalu mengatakan:

limit order pasti terisi.

Tetapi itu **belum tentu benar**.

Mengapa?

Karena kita tidak tahu:

- apakah order kita berada di depan atau belakang antrean,
- berapa banyak volume yang tersedia,
- apakah harga benar-benar menyentuh order,
- apakah fill hanya sebagian,
- bagaimana spread berubah,
- bagaimana latency memengaruhi order.

Jadi kita harus memodelkan:

price path + liquidity + order state.

F.2. Waktu menjadi variabel fundamental

Sebelumnya kita hanya memikirkan:

$$P_t.$$

Sekarang kita harus memiliki:

$$\boxed{D_t}$$

sebagai informasi pasar yang tersedia **tepat pada waktu t** .

Misalnya:

$$D_t = \{P_{\leq t}, V_{\leq t}, O_{\leq t}, F_{\leq t}\}.$$

Maka seluruh keputusan strategi harus memenuhi:

$$\boxed{a_t = \pi(D_t)}.$$

Tidak boleh:

$$a_t = \pi(D_{t+\Delta}).$$

Ini adalah prinsip anti-look-ahead bias.

F.3. Event-driven architecture

Kita definisikan event:

$$e_t.$$

Contohnya:

$$e_t \in \{MARKET_UPDATE, SIGNAL, ORDER_SUBMIT, ORDER_FILL, ORDER_CANCEL, SL_UPDATE, 1$$



Backtester kemudian menjalankan:

$$\boxed{e_1 \rightarrow e_2 \rightarrow e_3 \rightarrow \dots}$$

berdasarkan timestamp.

F.4. Market Event

Misalnya data datang:

$$e_t^{market}.$$

Informasinya:

$$M_t = \{bid_t, ask_t, last_t, volume_t, orderbook_t\}.$$

Dari sini kita memperbarui:

$$S_t.$$

F.5. State machine

Engine kita memiliki state global:

$$S_t = (MarketState, PortfolioState, OrderState, RiskState).$$

MarketState

$$M_t.$$

PortfolioState

$$\Pi_t.$$

OrderState

$$O_t.$$

RiskState

$$R_t.$$

F.6. Order state machine

Setiap order mempunyai state:

CREATED

↓

SUBMITTED

↓

PARTIALLY FILLED

↓

FILLED

atau:

CANCELLED

atau:

EXPIRED.

Ini penting karena order kita bukan hanya:

0/1.

F.7. Contoh

Misalkan:

$Q = 100$.

Kita ingin membeli:

100 *BTC*-equivalent units.



Namun hanya 40 yang tersedia pada harga tersebut.

Maka:

$$Q_{\text{filled}} = 40.$$

Remaining:

$$Q_{\text{remaining}} = 60.$$

Sehingga:

$$Q = Q_{\text{filled}} + Q_{\text{remaining}}.$$

F.8. Fill model

Untuk market order sederhana:

$$P_{\text{fill}} \approx P_{\text{ask}} + \text{slippage}.$$

Untuk LONG:

$$P_{\text{fill}} = P_{\text{ask}} + \delta.$$

Untuk SHORT:

$$P_{\text{fill}} = P_{\text{bid}} - \delta.$$

F.9. Slippage

Slippage dapat dimodelkan:

$$\delta_t = f(Q_t, \text{Depth}_t, \text{Volatility}_t, \text{Spread}_t).$$

Model sederhana:

$$\delta_t = \lambda \frac{Q_t}{\text{Depth}_t}.$$

Sehingga:

$$Q_t \uparrow \Rightarrow \delta_t \uparrow .$$

Ini sangat penting ketika nanti strategi kita mulai melakukan portfolio multi-asset.

F.10. Spread

Misalkan:

$$Bid_t = 100$$

dan:

$$Ask_t = 100.02.$$

Maka:

$$Spread_t = Ask_t - Bid_t = 0.02.$$

Relative spread:

$$s_t = \frac{Ask_t - Bid_t}{(Ask_t + Bid_t)/2}.$$

Strategi jangka pendek sangat sensitif terhadap:

$$s_t.$$

F.11. Transaction cost

Total biaya kita definisikan:

$$TC_t = Fee_t + SpreadCost_t + Slippage_t + MarketImpact_t.$$

Kemudian:

$$PnL_{net} = PnL_{gross} - TC.$$

Ini harus dimasukkan sejak awal.

Kalau tidak, strategi yang terlihat profitabel dapat berubah menjadi loss setelah biaya.

F.12. Market impact

Untuk ukuran order yang cukup besar:

$$\Delta P \propto Q^\alpha.$$

Model sederhana:

$$\text{Impact}(Q) = \eta \left(\frac{Q}{ADV} \right)^\alpha.$$

Dengan:

ADV = average daily volume.

Biasanya:

$$0 < \alpha < 1.$$

F.13. Kita sekarang bisa menghitung execution price

Misalnya LONG:

$$P_{\text{exec}} = Ask_t + \text{Impact}(Q) + \text{Slippage}_t.$$

Maka:

$$\text{Cost}_{\text{execution}} = Q(P_{\text{exec}} - P_{\text{mid}}).$$

F.14. Position accounting

Setelah fill, kita memiliki:

$$q_t$$

sebagai posisi.

Untuk LONG:

$$q_t > 0.$$

Untuk SHORT:

$$q_t < 0.$$

Untuk FLAT:

$$q_t = 0.$$

Nilai posisi:

$$V_t = q_t P_t.$$

F.15. Average entry price

Jika posisi masuk secara bertahap:

$$(q_1, P_1), (q_2, P_2), \dots$$

maka:

$$P_{\text{avg}} = \frac{\sum_i q_i P_i}{\sum_i q_i}.$$

Ini penting ketika terjadi partial fill.

F.16. Unrealized PnL

Untuk LONG:

$$PnL_t^{\text{unreal}} = q(P_t - P_{\text{avg}}).$$

Untuk SHORT:

$$PnL_t^{unreal} = q(P_{avg} - P_t),$$

dengan $q > 0$ sebagai absolute quantity.

Atau kita bisa menyatukannya menggunakan signed position:

$$PnL_t^{unreal} = q_t(P_t - P_{avg}).$$

F.17. Realized PnL

Ketika posisi ditutup:

$$PnL^{realized} = q_{closed}(P_{exit} - P_{entry}) - TC.$$

Kemudian:

$$Equity_t = Equity_0 + \sum_{\tau \leq t} PnL_{\tau}^{realized} + PnL_t^{unrealized}.$$

F.18. Risk unit

Kita sebelumnya menetapkan:

$$R = \$1.$$

Maka setiap trade harus mempunyai:

$$Risk_i \leq R.$$

Misalnya entry:

$$P_e = 100.$$

SL:

$$P_{SL} = 99.$$

Risk per unit:

$$r = 1.$$

Untuk:

$$R = \$1,$$

quantity:

$$q = \frac{R}{|P_e - P_{SL}|}.$$

Sehingga:

$$q = 1.$$

F.19. Contoh lain

Entry:

$$P_e = 100.$$

SL:

$$P_{SL} = 98.$$

Maka risk per unit:

$$2.$$

Jika:

$$R = \$1,$$

maka:

$$q = \frac{1}{2}.$$

Jika SL terkena:

$$Loss \approx \$1$$

sebelum biaya.

F.20. Tetapi kita harus memperhitungkan gap

Misalkan:

$$SL = 98.$$

Namun harga melompat:

$$100 \rightarrow 95.$$

Maka actual loss:

$$100 - 95 = 5.$$

Bukan:

$$100 - 98 = 2.$$

Jadi:

$$\boxed{Risk_{\text{realized}} \neq Risk_{\text{planned}}.}$$

Untuk saham, gap risk sangat penting.

Untuk crypto 24/7, gap biasanya berbeda karakter tetapi tetap dapat muncul pada perubahan likuiditas ekstrem.

F.21. Maka kita membutuhkan risk multiplier

Misalnya:

$$q = \frac{R}{E[Loss|SL]}.$$

Bukan sekadar:

$$\frac{R}{SL - distance}.$$

Jika expected execution loss:

$$E[Loss] = 1.25$$

maka:

$$q = \frac{1}{1.25} = 0.8.$$

F.22. Order lifecycle

Sekarang mari kita ikuti satu trade.

t₀

Model menghasilkan:

$$x^* = 1.$$

Artinya LONG.

t₁

Trade constructor menghasilkan:

$$Entry = 100$$

$$SL = 99$$

$$TP = 102.$$

t₂

Order dikirim.

State:

PENDING.



t₃

Market:

$$Bid = 99.99$$

$$Ask = 100.01.$$

Order belum filled.

t₄

Market berubah.

Model baru:

$$EV_{\text{entry}} < \theta.$$

Maka:

CANCEL.

Trade tidak pernah terjadi.

Ini harus tercatat sebagai cancellation, bukan loss.

F.23. Contoh kedua

Order filled:

$$P_e = 100.$$

SL:

$$99.$$

TP:

$$102.$$

Kemudian:

$$P_t = 100.8.$$

Model menghitung:

$$J(HOLD) > J(EXIT).$$

Tetap HOLD.

F.24. Kemudian

Harga:

$$101.2.$$

Model menghitung:

$$SL^* = 100.7.$$

Maka:

$$SL : 99 \rightarrow 100.7.$$

Sekarang maximum planned loss bukan lagi:

$$1R.$$

Tetapi posisi sudah memiliki locked profit:

$$0.7R.$$

F.25. Kemudian

Harga:

$$102.5.$$

Model mengatakan:

$$TP^* = 104.$$



Maka:

$TP : 102 \rightarrow 104.$

F.26. Kemudian harga turun

$102.7 \rightarrow 101.9 \rightarrow 100.7.$

SL terkena.

Realized PnL:

$+0.7R$

sebelum biaya.

Inilah contoh konkret dari **dynamic position management**.

F.27. Data yang diperlukan

Sekarang kita harus menentukan data minimum.

Untuk tahap awal:

Asset data

OHLCV.

Market factors

Untuk saham internasional:

DXY, Gold, SPX, VIX, Treasury yields.

Untuk crypto:

BTC, ETH, BTC.D, ETH/BTC.

Kemudian:

asset-specific data.



F.28. Untuk strategi short-term, timeframe harus multi-resolution

Saya menyarankan kita tidak hanya memiliki satu timeframe.

Misalnya:

Slow

$1D$

untuk regime.

Medium

$1H$

untuk signal.

Fast

$5m$

untuk entry.

Jika data tersedia:

Microstructure

$1m$

atau bahkan tick/order book.

Sehingga:

$$S_t = (S_t^{1D}, S_t^{1H}, S_t^{5m}, S_t^{1m})$$

F.29. Mengapa multi-timeframe?

Misalnya:

Daily:

Trend = UP.

Hourly:

Momentum = UP.

5-minute:

Pullback.

1-minute:

Bid imbalance > 0.

Maka semua layer konsisten.

Sebaliknya:

Daily:

Trend = DOWN.

5-minute:

Momentum = UP.

Mungkin LONG jangka pendek tetap bisa dilakukan, tetapi confidence harus lebih rendah.

F.30. Data leakage menjadi lebih rumit

Misalkan kita sedang berada di:

10 : 00.

Candle 1H:

10 : 00 – 11 : 00.

Kita **tidak boleh menggunakan close 1H** pada 10:15 karena candle belum selesai.

Kita hanya boleh menggunakan:

$$Open_{10:00}, High_{\leq 10:15}, Low_{\leq 10:15}, Volume_{\leq 10:15}.$$

Ini kesalahan yang sangat umum dalam backtest.

F.31. Point-in-time data

Prinsipnya:

$$Feature_t = f(\text{Data available at } t).$$

Bahkan untuk fundamental data:

jangan menggunakan laporan yang baru dipublikasikan pada:

$$t + 2$$

seolah-olah sudah diketahui pada:

$$t.$$

Untuk strategi kita, ini menjadi prinsip wajib.

F.32. Walk-forward framework

Sekarang kita masuk ke struktur validasi.

Misalnya:

$$T_1 = \text{training}$$

$$V_1 = \text{validation}$$

$$O_1 = \text{out-of-sample.}$$

Kemudian rolling:

$$T_2 \rightarrow V_2 \rightarrow O_2.$$

Sehingga:

Train → Validate → Trade unseen data

berulang.

F.33. Mengapa tidak random train-test split?

Karena data finansial adalah time series.

Jika kita melakukan:

random train/test split

maka:

t_{future}

bisa masuk training sementara:

t_{past}

masuk test.

Ini merusak temporal causality.

Jadi:

No random shuffling.

F.34. Parameter yang harus di-fit

Banyak sekali parameter kita:

$$\theta = \{\lambda, \gamma, \beta, k_{SL}, k_{TP}, \kappa, threshold, lookback, risk\ budget, \dots\}.$$

Masalahnya:

$|\theta|$

bisa sangat besar.

Jika kita terlalu agresif melakukan optimization:

$$\theta^* = \arg \max_{\theta} Performance(\theta)$$

pada historical data, kita bisa mendapatkan:

Overfitting.

F.35. In-sample performance bukan bukti

Misalkan:

$$Sharpe = 4.8$$

pada training data.

Itu belum berarti apa-apa.

Kita lebih tertarik pada:

$$Sharpe_{OOS}.$$

Dan bahkan:

$$Sharpe_{OOS}$$

yang tinggi sekali juga harus dicurigai jika parameter terlalu banyak.

F.36. Parameter stability

Kita ingin melihat:

$$Performance(\theta)$$

tidak hanya bagus di satu titik.

Misalnya:

$$k_{SL} = 1.8$$

menghasilkan Sharpe:

1.7.

Tetapi:

$$k_{SL} = 1.7 \rightarrow 1.6$$

juga menghasilkan:

1.65, 1.61.

Ini bagus.

Sebaliknya:

$$k_{SL} = 1.80 \rightarrow \text{Sharpe} = 2.5$$

tetapi:

$$k_{SL} = 1.79 \rightarrow 0.3.$$

Itu tanda kuat overfitting.

F.37. Kita perlu baseline

Ini sangat penting untuk proyek quantum kita.

Kita tidak boleh hanya mengatakan:

"QAOA menghasilkan profit."

Kita harus membandingkan:

QAOA vs Classical Optimizer.

Minimal:

Baseline 1

Equal weighting.

Baseline 2

Greedy selection.

Baseline 3

Classical QUBO solver.

Misalnya:

Simulated Annealing.

Baseline 4

MILP / exact solver untuk problem kecil.

Baseline 5

Heuristic portfolio selection.

F.38. Baru kemudian QAOA

Misalnya:

QUBO

yang sama diberikan kepada:

$\left\{ \begin{array}{l} \textit{Exact} \\ \textit{Simulated Annealing} \\ \textit{Greedy} \\ \textit{QAOA} \end{array} \right.$

Semua menghasilkan:

x^* .

Kemudian masing-masing diteruskan ke **execution simulator yang sama**.

Ini sangat penting.

Kalau execution model berbeda, perbandingan tidak fair.

F.39. Eksperimen yang benar

Kita definisikan:

$$A = \{A_{greedy}, A_{SA}, A_{exact}, A_{QAOA}\}.$$

Untuk setiap:

$$A_i,$$

gunakan:

$$Data, Cost, Risk, Execution$$

yang sama.

Kemudian ukur:

$$Performance(A_i).$$

F.40. Metrics

Jangan hanya menggunakan profit.

Minimal:

$$Total\ Return$$

$$CAGR$$

$$Volatility$$

$$Sharpe$$

$$Sortino$$

$$Max\ Drawdown$$

$$Calmar$$



Win Rate

Profit Factor

Turnover

Transaction Cost.

F.41. Sharpe Ratio

Dengan return:

$$r_t,$$

maka:

$$\boxed{Sharpe = \frac{E[r - r_f]}{\sigma(r - r_f)} \sqrt{K}}.$$

Untuk intraday, K harus dipilih sesuai frekuensi sampling secara konsisten.

F.42. Maximum Drawdown

Equity:

$$E_t.$$

Running maximum:

$$M_t = \max_{\tau \leq t} E_\tau.$$

Drawdown:

$$DD_t = \frac{E_t - M_t}{M_t}.$$

Maka:

$$MDD = \min_t DD_t.$$

F.43. Tetapi untuk strategi kita ada metric tambahan

Karena kita menggunakan optimizer kombinatorial:

Solution quality

$$Gap = \frac{C(x_{QAOA}) - C(x^*)}{|C(x^*)|}.$$

Jika exact solution tersedia.

F.44. Quantum-specific metrics

Kita bisa mengukur:

Approximation Ratio.

Untuk maximization:

$$AR = \frac{C(x_{QAOA})}{C(x^*)}.$$

Kemudian:

QAOA runtime

number of circuit evaluations

depth

shots.

F.45. Ini penting untuk tesis/proyekmu

Karena kalau QAOA menghasilkan:

$$Return = 18\%$$

sementara classical:

$$Return = 16\%,$$

itu menarik.

Tetapi kalau QAOA membutuhkan:

$$10^6$$

circuit evaluations dan classical solver hanya:

$$10^3,$$

kita tidak bisa hanya mengatakan:

QAOA lebih baik.

Kita harus mempertimbangkan:

$$Performance + Computational Cost.$$

F.46. Kita dapat membuat objective eksperimental

Misalnya:

$$Score = Sharpe - \lambda MDD - \eta TC - \xi ComputationalCost.$$

Atau mempertahankan metrics terpisah agar tidak mencampur interpretasi.

Untuk penelitian, saya lebih menyukai **menyimpan semua metrik secara terpisah**, lalu membandingkan Pareto frontier.

F.47. Pareto frontier

Misalnya kita punya:

$$x - axis = Sharpe$$

dan:

$$y - axis = ComputationalCost.$$

Kita ingin mencari solusi yang tidak didominasi:

$$Pareto\ optimal.$$

Ini sangat relevan untuk membandingkan:

$$QAOA$$

dengan:

$$classical.$$

F.48. Stress testing

Backtest normal belum cukup.

Kita harus menguji:

Spread meningkat

$$Spread \rightarrow 2Spread.$$

Slippage meningkat

$$Slippage \rightarrow 3Slippage.$$

Latency meningkat

$$Latency \rightarrow 5 \times .$$

Volatility meningkat

$$\sigma \rightarrow 2\sigma.$$

Liquidity turun

$$Depth \rightarrow 0.5Depth.$$

Jika strategi langsung runtuh, kita tahu strategi terlalu rapuh.

F.49. Monte Carlo trade reshuffling

Misalkan kita memiliki trade returns:

$$r_1, r_2, \dots, r_N.$$

Kita dapat melakukan bootstrap:

$$r_1^*, \dots, r_N^*.$$

Kemudian membentuk equity curve baru.

Ulangi:

$$B = 10,000$$

kali misalnya.

Kita mendapatkan distribusi:

$$MDD^*,$$

$$Return^*,$$

$$Sharpe^*.$$

Bukan hanya satu angka.

F.50. Probability of ruin

Jika risk per trade:

$$R = \$1,$$

dan capital:

$$C_0.$$

Kita dapat mengestimasi probabilitas:

$$P(C_t < C_{\min}).$$

Ini jauh lebih informatif daripada hanya melihat CAGR.

F.51. Regime stress test

Karena kita memiliki:

$$F_t = (DXY, Gold, SPX, VIX, BTC.D, \dots),$$

kita dapat membagi data:

Regime 1 = low volatility

Regime 2 = high volatility

Regime 3 = risk-on

Regime 4 = risk-off.

Kemudian:

$$Performance(A|Regime_k).$$

Ini memungkinkan kita mengetahui:

kapan QAOA-based strategy bekerja dan kapan tidak.

F.52. Ini membawa kita pada konsep yang lebih menarik

Kita sebenarnya tidak hanya menguji:

QAOA vs Classical.

Kita menguji:

Algorithm × Market Regime × Execution Condition.

Misalnya:

Regime	QAOA	Classical
Low vol	1.4 Sharpe	1.3
High vol	0.8	0.7
Risk-on	1.8	1.5
Risk-off	0.5	0.6

Mungkin QAOA unggul pada regime tertentu tetapi tidak pada lainnya.

Itu justru hasil penelitian yang jauh lebih menarik daripada klaim "quantum selalu lebih baik".

F.53. Arsitektur software yang saya sarankan

Saya akan membagi project menjadi:

```
TradeProject/
|
├─ data/
|   ├── raw/
|   ├── processed/
|   └─ features/
|
├─ market/
|   ├── market_state.py
|   ├── orderbook.py
|   └─ factors.py
|
├─ forecasting/
|   ├── returns.py
|   ├── volatility.py
|   ├── correlation.py
|   └─ regime.py
|
├─ optimization/
|   ├── qubo.py
|   ├── ising.py
|   ├── classical.py
|   └─ qaoa.py
|
├─ execution/
|   ├── order.py
|   ├── fill.py
|   ├── slippage.py
|   └─ transaction_cost.py
|
├─ portfolio/
|   ├── position.py
|   ├── pnl.py
|   └─ risk.py
|
├─ strategy/
|   ├── entry.py
|   ├── position_manager.py
|   └─ exit.py
|
├─ backtest/
|   └─ engine.py
```

```
|   ├── events.py
|   └── simulator.py
|
|── validation/
|   ├── walk_forward.py
|   ├── stress_test.py
|   └── monte_carlo.py
|
└── experiments/
    ├── qaoa_vs_classical.py
    └── regime_analysis.py
```

Ini jauh lebih baik daripada satu file besar yang berisi seluruh algoritma.

F.54. Event loop utama

Secara matematis:

$$S_{t+1} = T(S_t, e_t, a_t)$$

dengan:

T

sebagai transition function.

Secara konseptual:

```
initialize()

for event in chronological_events:

    update_market(event)

    update_orders(event)

    process_fills(event)

    update_positions(event)

    update_pnl(event)

    update_risk(event)

    if decision_time(event):

        state = build_state()

        forecast = model(state)

        optimization = optimizer(forecast)

        orders = strategy(optimization)

        submit_or_modify(orders)

record_state()
```

F.55. Ini adalah "digital twin" kecil dari trading system kita

Kita sebenarnya sedang membuat:

Market Simulator

yang memungkinkan kita bertanya:

"Apa yang akan terjadi jika algoritma ini benar-benar beroperasi?"

Tanpa harus mempertaruhkan uang nyata.

F.56. Namun ada satu hal yang belum kita selesaikan

Data order book historis.

Jika kita ingin benar-benar melakukan:

order placement + queue position + partial fill,

OHLCV saja tidak cukup.

Idealnya kita memiliki:

L2/L3 orderbook.

Misalnya:

$$Bid : \begin{cases} 100.00 : 100 \\ 99.99 : 150 \\ 99.98 : 300 \end{cases}$$

dan:

$$Ask : \begin{cases} 100.01 : 120 \\ 100.02 : 200 \\ 100.03 : 350 \end{cases}$$

Dari sini kita dapat menghitung:

$$OBI = \frac{V_{bid} - V_{ask}}{V_{bid} + V_{ask}}.$$

F.57. Order Book Imbalance

$$OBI_t = \frac{\sum_{i=1}^k B_i - \sum_{i=1}^k A_i}{\sum_{i=1}^k B_i + \sum_{i=1}^k A_i}.$$

Dengan:

$$-1 \leq OBI \leq 1.$$

Jika:

$$OBI \rightarrow 1,$$

bid dominance.

Jika:

$$OBI \rightarrow -1,$$

ask dominance.

Ini bisa menjadi feature untuk:

$$\mu_t.$$

F.58. Tetapi kita jangan langsung melompat ke L2

Untuk **versi pertama sistem**, saya justru menyarankan tiga tingkat fidelity.

Level 1 — Research backtest

Gunakan:

$$OHLCV.$$

Model:

$$spread, slippage$$

secara parametrik.

Level 2 — Execution-aware backtest

Tambahkan:

$$Bid/Ask, volume, trade data.$$

Level 3 — Microstructure simulator

Tambahkan:

L2/L3, queue, partial fill, latency.

Dengan demikian kita tidak membuat sistem terlalu kompleks sebelum fondasinya tervalidasi.

F.59. Urutan implementasi yang saya rekomendasikan

Jangan langsung membuat semuanya.

Kita lakukan:

F_1

Historical data engine.

↓

F_2

Event engine.

↓

F_3

Order simulator.

↓

F_4

Position/PnL engine.

↓

F_5

Risk engine.

↓

F_6

Dynamic position management.

↓

F_7

Walk-forward validation.

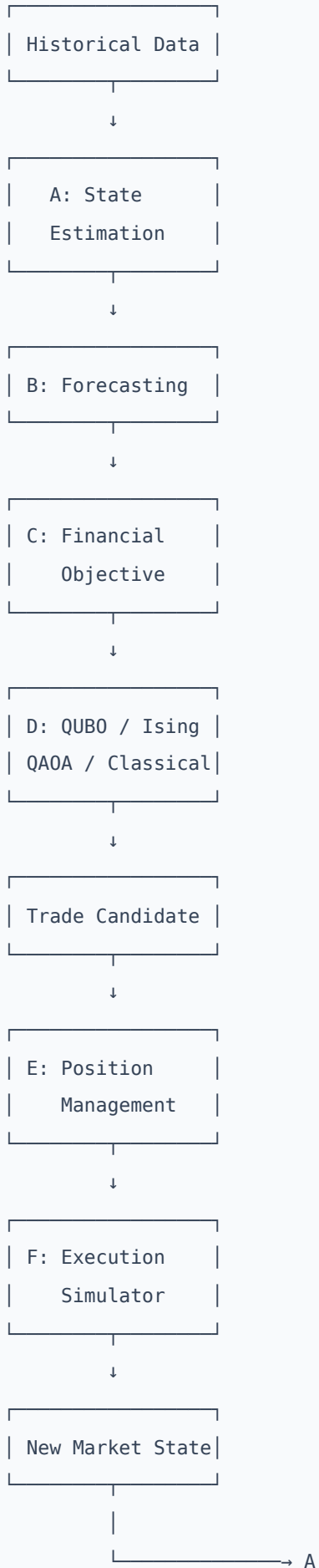
↓

F_8

QAOA vs classical experiment.

F.60. Hubungan Tahap A–F

Sekarang keseluruhan sistem kita mulai terlihat sangat jelas:



Jadi sistemnya **closed-loop**.

F.61. Satu prinsip yang saya ingin kita pegang

Jangan mengoptimalkan:

historical profit.

Yang harus kita optimalkan adalah:

robust decision-making under uncertainty.

Karena tujuan akhirnya bukan:

menemukan parameter yang paling cocok dengan masa lalu.

Tetapi:

menemukan algoritma yang tetap masuk akal ketika kondisi pasar berubah.

F.62. Dan di sinilah proyek ini mulai menjadi penelitian quantitative trading yang serius

Kita nantinya bisa menguji beberapa hipotesis:

H1

Apakah factor-aware forecasting lebih baik daripada price-only forecasting?

$$H_1 : \text{Performance}(X, F) > \text{Performance}(X).$$

H2

Apakah combinatorial optimization lebih baik daripada greedy selection?

$$H_2 : J(x_{\text{QUBO}}) > J(x_{\text{greedy}}).$$

H3

Apakah QAOA mendekati solusi classical/exact?

$$H_3 : \quad AR_{\text{QAOA}} \approx 1.$$

H4

Apakah dynamic position management meningkatkan risk-adjusted return?

$$H_4 : \quad \textit{Sharpe}_{\text{dynamic}} > \textit{Sharpe}_{\text{static}}.$$

H5

Apakah keunggulan QAOA tetap ada setelah transaction cost dan slippage?

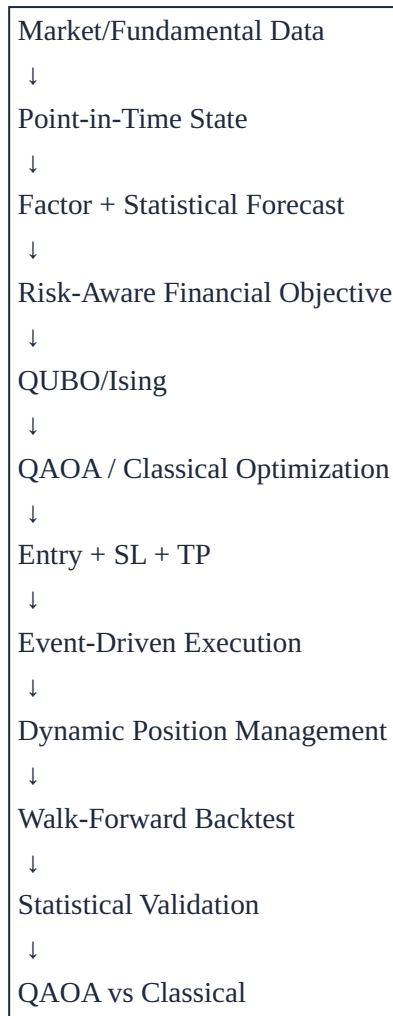
$$H_5 : \quad J_{\text{QAOA}}^{\text{net}} > J_{\text{classical}}^{\text{net}}.$$

H5 justru merupakan salah satu eksperimen terpenting.

Karena sebuah metode bisa terlihat unggul sebelum biaya eksekusi, tetapi keunggulannya hilang setelah biaya dimasukkan.

F.63. Posisi kita sekarang

Dengan Tahap F selesai secara desain, pipeline kita sudah menjadi:



Dan ada satu hal yang menurut saya **sengaja harus kita tahan dulu**: jangan langsung menghubungkan ini ke Binance atau broker nyata.

Tahap berikutnya, **Tahap G**, sebaiknya kita gunakan untuk membangun **mathematical specification dari seluruh objective function dan QUBO final**—termasuk bagaimana **expected return, covariance, factor exposure (DXY/gold/SPX/BTC/ETH), transaction cost, turnover, position constraints, dan risk budget** semuanya masuk ke satu formulasi:

$$\min_{x \in \{0,1\}^N} x^T Q x$$

lalu diturunkan secara eksplisit sampai menjadi **Ising Hamiltonian**

$$H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j$$

yang benar-benar bisa kita masukkan ke **QAOA**.

Itu akan menjadi titik di mana seluruh pembahasan quantitative finance kita bertemu langsung dengan quantum computing yang sudah kamu pelajari.